

Un cliente, dos proveedores

PF1822 · Especialización en Desarrollo con IA

Módulo 2 · MA04576 · Introducción a modelos generativos y consumo por API

MODALIDAD

Grupal, de 4 a 5 personas, en micro salas durante la sesión sincrónica.

APRENDIZAJE ESPERADO 2

Utilizar un modelo de IA a través de API REST utilizando solicitudes HTTP y autenticación, basándose en la documentación oficial de OpenAI y Hugging Face.

CRITERIOS DE EVALUACIÓN

- 2.1 Explica la estructura y principales secciones de los endpoints de generación y embeddings de acuerdo con la documentación oficial de OpenAI y Hugging Face.
- 2.2 Implementa llamadas HTTP GET y POST a endpoints de completions y embeddings para el consumo de servicios, de acuerdo con la documentación oficial.
- 2.3 Implementa una función o clase modular en Python que encapsula la llamada al API y permite reutilizarla con distintos parámetros, documentada con docstrings y pruebas unitarias simples.

Contexto

El equipo ya diseñó la arquitectura de la plataforma de soporte de Nube Sur. Ahora toca programar la pieza que conversa con los modelos.

Problema

Nube Sur no quiere depender de un solo proveedor de modelos: el mes pasado otra empresa del rubro quedó sin servicio un día entero por una caída de su proveedor. Además, cada desarrollador escribe sus propias llamadas a la API, con claves copiadas en el código y sin manejo de errores.

Solución

Un paquete de Python, `nubesur_ia`, con una misma interfaz para generar texto y obtener embeddings con OpenAI o con Hugging Face. Para cambiar de proveedor basta con cambiar una variable de entorno.

Desarrollo

- 1. Documentación (criterio 2.1).** Revisen la documentación oficial de ambos proveedores y escriban en el `README.md` una tabla con los endpoints de generación y de embeddings de cada uno: URL, método, autenticación, cuerpo de la solicitud y ubicación del resultado.
- 2. Interfaz común.** Definan una clase base `Proveedor` con dos métodos: `generar(mensajes, temperature=0.2, max_tokens=200)`, que devuelve el texto y los tokens usados, y

`embeddings(textos)`, que devuelve una lista de vectores.

3. **Implementación (criterio 2.2).** Programen `ProveedorOpenAI` y `ProveedorHuggingFace` con `requests` o `httpx`. Cada uno lee su clave desde una variable de entorno (`OPENAI_API_KEY` y `HF_TOKEN`), usa un tiempo de espera y maneja con un mensaje claro los errores 401 y 429 y los tiempos de espera agotados.
4. **Selección del proveedor.** Una función `crear_cliente()` devuelve el proveedor indicado en la variable de entorno `PROVEEDOR_IA` (`openai` o `huggingface`).
5. **Pruebas y docstrings (criterio 2.3).** Documenten cada clase y método con docstrings y escriban al menos **cuatro pruebas unitarias** que se ejecuten sin conexión, con la API simulada: una respuesta correcta de cada proveedor, un error 401 y un error 429.
6. **Revisión de código.** Cada integrante revisa el código de otro (por *pull request* o con comentarios) y deja al menos una sugerencia. Registren qué cambios se aceptaron.
7. **Demostración.** Envíen el mismo mensaje con los dos proveedores y comparen la respuesta y los tokens usados.

Roles del equipo

Roten los roles respecto del ABPRO anterior:

- **Coordinador/a:** integra el código en el repositorio y cuida el tiempo.
- **Responsable de OpenAI:** programa `ProveedorOpenAI`.
- **Responsable de Hugging Face:** programa `ProveedorHuggingFace`.
- **Responsable de pruebas:** escribe las pruebas unitarias y las simulaciones.
- **Documentador/a y portavoz:** escribe el `README.md` y presenta la demostración.

Entrega

Una entrega por equipo en esta Tarea:

- el enlace al repositorio (GitHub o equivalente) con el paquete, las pruebas y el `README.md`;
- una captura de la salida de `pytest`;
- la comparación de la demostración;
- el registro de la revisión de código;
- los nombres de los integrantes con su rol.

Ninguna clave puede aparecer en el repositorio: usen un archivo `.env` fuera del control de versiones y un `.env.example` sin valores.